UI Modernization Demo Script - Bob Java Modernization Workflow

Section 1 — Setting the Stage: The Legacy Application

Verbiage

"This application Simple Pharmacy Dashboard is a traditional Java web application built on Apache Struts 2, running on IBM WebSphere. Every user interaction triggers a full round-trip to the server: the server renders a HTML page from a JSP template and sends the whole thing back to the browser. There is no API, no modern frontend framework, and no separation between the UI and the business logic. It works, but is rigid, slow to change, and difficult to maintain. Let's see what that same application looks like after running it through the Java UI Modernization workflow in Bob."

Section 2 — The Modernization Workflow in Bob

Verbiage

"After launching the Java Modernization workflow directly inside Bob, we choose 'UI Modernization' then continue, and have Bob regenerate Architecture analysis. The entire codebase is assessed across every action class, JSP view, and model producing three output artifacts: a detailed architecture document, a Mermaid architecture diagram, and a JSON file enumerating the modernization options. Then to confirm our framework choices: React with the Carbon Design System on the frontend, with Jakarta EE REST services on the backend — and Bob executed the full migration end-to-end."

Steps

- Select 'UI Modernization' and continue
- Regenerate Architecture analysis
- Open `architecture-diagram.mmd` showing the application's component relationships
- Select Frontend framework
- Select Frontend design system
- Select Backend framework
- Frontend root folder path
- Backend root folder path
- Select: Setup Project

Section 3 — The New Backend: Jakarta EE REST API

Verbiage

"The first major output is a brand-new Jakarta EE backend. Bob took the existing in-memory data model across medicines, prescriptions, and orders, and wrapped it in a clean REST API using JAX-RS. It rapidly scaffolds the new project in parallel creating essential configuration files such as pom.xml, server.xml, and project structure artifacts. When encountering environment constraints, like restricted file creation for .gitignore, Bob adapts seamlessly by executing shell

commands to ensure the file is still correctly provisioned. It then proceeds to faithfully migrate core components, copying model and repository classes without modification to preserve business logic integrity, while simultaneously generating the new Jakarta EE source files. We can confirm the application us running.”

Steps

- Establish a new Jakarta EE backend foundation
- Scaffold the project structure in parallel
- Handle environment constraints proactively, using shell commands
- Migrate core domain logic intact
- Generate new Jakarta EE source files alongside the migrated components
- Confirm application backend is running correctly

Section 4 — The New Frontend: React + Carbon Design System

Verbiage
"Now for the part that's most visible to users. The frontend is a React TypeScript single-page application built with Vite, using IBM's own Carbon Design System as the UI component library.

Bob analyzes the project layout and backend configuration, extracting critical details like server port, API context path, and CORS behavior. Based on these insights, Bob defines a consistent development environment, aligning the backend on port 9080 and provisioning the frontend on port 3000. It then constructs a structured execution plan and scaffolds a modern React + TypeScript application using Vite, leveraging fast build tooling and best-practice defaults. Every command is executed within a controlled approval flow, maintaining transparency and governance. Carbon gives us a professional, accessible, enterprise-grade look and feel out of the box consistent typography, color tokens, grid layout, and interactive components like data tables, modals, and notifications. The result is a fully aligned frontend foundation, ready to integrate cleanly with the newly modernized Jakarta EE backend.

Steps

- Initiate frontend setup by preparing to scaffold a React + TypeScript application using Vite
- Explore existing project structure, reviewing both backend and frontend directories
- Inspect backend configuration by analyzing the pom.xml to extract key details
- Establish architectural alignment
- Create a structured task plan to guide the frontend scaffolding
- Execute project scaffolding command
- Request execution approval, ensuring controlled and auditable command execution within the development workflow
- Confirm application starts correctly

Section 5 — Dashboard Page

Spoken verbiage:
"With a complete understanding of both the frontend structure and backend API contracts, Bob moves into full UI implementation mode. It begins by analyzing existing stubs and resource definitions to ensure every component aligns precisely with available endpoints. From there, Bob executes a highly efficient, parallelized build process, first generating a suite of shared components and utilities that standardize layout, loading states, and error handling across the application. It then layers on higher-level functionality, introducing global elements like toast notifications before progressing to feature-rich pages such as the dashboard with dynamic stat cards. Bob then re-executes the build process, successfully achieving a clean compilation with zero TypeScript errors. The only remaining outputs are minor, non-blocking warnings related to font resolution and bundle sizing, confirming that the application is now stable, optimized, and ready for deployment. By systematically expanding into additional views like prescriptions, Bob ensures the entire frontend is cohesive, reusable, and tightly integrated with the backend, resulting in a modern, scalable user interface built with consistency and speed."

Steps

- Analyze existing frontend and backend artifacts
- Inspect backend resource definitions, identifying exact API endpoints
- Establish a complete system view, combining UI structure and API contracts
- Define a structured implementation plan, outlining components, hooks, and pages to be built
- Generate shared UI components in parallel, including reusable elements
- Build higher-level UI components, including the ToastContainer for notifications
- Develop feature pages iteratively, starting with the DashboardPage (including stat cards and key visuals)
- Extend implementation to additional pages, such as prescriptions and other domain-specific views

Section 6 — Frontend and Backend Integration

Spoken verbiage:
"With all components and integrations in place, Bob completes the final refinement cycle, ensuring the application is fully functional, cohesive, and production-ready. After thoroughly analyzing the existing codebase, it resolves styling conflicts, fills functional gaps, and introduces key features such as Prescription and Medicine detail pages. Navigation flows are enhanced across all list views, while routing is updated to seamlessly connect every part of the application. Bob then executes a full production build, identifying and resolving TypeScript issues until achieving a clean compile with zero errors. Any remaining messages are correctly recognized as non-blocking optimizations rather than failures. Finally, Bob validates the application end-to-end, confirming that all critical user journeys from dashboard actions to detailed views and order processing work as expected. The result is a fully integrated, stable, and modern React application, ready for real-world use and deployment."

Steps

- Initiate integration and refinement phase

- Analyze existing codebase, including pages, components, and configuration files to identify gaps and inconsistencies
- Prioritize and resolve key issues, such as styling conflicts
- Implement missing detail pages, including Prescription Detail and Medicine Detail views
- Update application routing, modifying App.tsx to expose all new detail routes
- Enhance navigation across the app
- Refine dashboard interactions
- Execute production build, compiling the application to validate type safety and runtime readiness
- Validate final build output, confirming zero TypeScript errors

Section 7 — Validate the Dashboard UI

Spoken verbiage:
"Dashboard, Prescriptions, Orders and Medicines pages all load. Pages are thin, they consume hooks and render Carbon components. Shared utilities cover validation, formatting, data transformation, and logging. Pages load successfully, with smooth transitions across each, and all functionality working. We can confirm everything is working"

Steps

- Verify end-to-end functionality, running the app locally and testing critical user journeys across dashboard, prescriptions, orders, and medicines

Section 8 — Task Completion Summary

Spoken verbiage:
"With all development and integration steps complete, Bob enters the final validation stage, ensuring the entire modernization effort meets production standards. It confirms architectural improvements such as clear separation of concerns, alongside significant gains in code quality and maintainability. Dependencies are verified to be fully up to date, and every major deliverable has been successfully implemented and integrated. Bob then performs a full validation pass, confirming that the build succeeds with zero errors and that all validation checks are passed. To complement this, it generates a Mermaid-based visualization that clearly illustrates the modernization journey from start to finish, providing both technical and visual traceability."

Steps

- Initiate final validation phase
- Verify dependency upgrades
- Architecture analysis and documentation
- Fully implemented and integrated frontend components and pages
- Confirm that all tests and checks pass successfully with no errors
- Generate modernization visualization, producing a Mermaid diagram
- Outline suggested next steps

Section 9 — Closing Summary

With a clean build, successful validation, and complete transparency into the process, the application stands as fully modernized, stable, and ready for deployment, with clear next steps for review and continued use. So to summarize what the workflow delivered: we took a monolithic Java Struts application running on WebSphere and produced a modern, two-tier system, a Jakarta EE REST backend on Liberty and a React Carbon frontend with zero manual scaffolding. Bob analyzed the source code, made framework recommendations, ran the migration, fixed build errors, and validated the result. The application is now easier to extend, and easier to test."